



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3133 - HIGH PERFORMANCE DATA PROCESSING

SECTION 02

PROJECT 2

Real-Time Sentiment Analysis using Apache Spark and Kafka

LECTURER: DR. SEAH CHOON SEN

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164
LUBNA AL HAANI BINTI RADZUAN	A23CS0107
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156
NURAI SYAH BINTI MOHD ZIKRE	A23CS0160

Table of Contents

1.0 Introduction	4
1.1 Background	4
1.2 Objectives	5
1.3 Scope	5
2.0 Data Acquisition & Preprocessing	6
2.1 Data Source Justification	6
2.2 Data Collection	7
2.3 Sentiment Labeling	8
2.4 Preprocessing Pipeline	8
2.4.2 Step 2 - Noise Removal	9
2.4.3 Step 3 - Tokenization	9
2.4.4 Step 4 - Stop Word Removal	9
2.4.5 Step 5 - Lemmatization	9
2.4.6 Step 6 - Dataset Export	10
2.5 Final Dataset Summary	10
3.0 Sentiment Model Development	11
3.1 Model Development and Training Process	11
3.1.1 Machine Learning Model (Naive Bayes)	11
3.1.2 Deep Learning Model (LSTM)	12
3.2 Evaluation and Model Selection	13
4.0 Apache System Architecture	14
4.1 Data Pipeline Architecture	14
4.2 Apache Kafka	15
4.3 Apache Spark Structured Streaming	15

4.4 Elasticsearch	16
5.0 Analysis & Results	17
5.1 Model Findings	17
5.2 Pipeline	19
5.3 Dashboard	21
6.0 Comparison & Optimization	27
6.1 Methodology	27
6.2 Results	27
6.3 Optimization Opportunities	29
7.0 Conclusion & Future Work	30
7.1 Conclusion	30
7.2 Future Work	30
References	32
Appendix	33

1.0 Introduction

The main idea of this project is to implement a real-time sentiment analysis data pipeline for Malaysian mobile app reviews. It links the development of Malaysia's digital economy to the requirement for real-time feedback with Natural Language Processing (NLP) and stream processing tools. The following sections outline the background for the study, the specific objectives that will be addressed and the overall scope of the project.

1.1 Background

The expansion of digital platforms in recent years has created an exceptional amount of text data generated by users. Users write reviews about a product, service or news in the apps they use, post on social media and comment on stories in the news. Companies that can make sense of this data in real-time have a huge competitive advantage, as they can act fast on customer concerns, manage their brand's reputation and take data-driven decisions.

Sentiment analysis or opinion mining is a subfield of Natural Language Processing (NLP) categorizing and recognizing the emotional content of a piece of text (Pang & Lee, 2008). The typical sentiment analysis system classifies text into three categories: Positive, Neutral, and Negative. This is a powerful tool when used on a large scale and in real time, for businesses and researchers alike.

Malaysia's digital economy is a rapidly developing economy in Southeast Asia. The report on the economy SEA 2023 states that ecommerce, ride-hailing and digital payment platforms are expected to be the primary drivers of internet economy growth in Malaysia, reaching USD 35 billion by 2025 (Hoppe et al., 2023). The apps that provide to millions of Malaysian users every day like Touch 'n Go eWallet, Grab and foodpanda have built up a massive pile of public user reviews over at the Google Play Store. The sentiment analysis of these reviews in real time offers firsthand insight into users' satisfaction, service quality, and what needs to be improved.

Traditional batch processing methods look at data once it has been gathered and stored, meaning there's a latency period. Unlike real-time stream processing systems, which analyze data as it is received, traditional batch processing systems work on data batches to generate insights. Apache

Kafka and Apache Spark are examples of technologies that have emerged as the industry standards for creating scalable, fault tolerant real-time data pipelines. This project will utilize these technologies along with current NLP techniques to build an end-to-end sentiment analysis system on data regarding Malaysia.

1.2 Objectives

The goals of this project sentiment analysis are:

- To gather and preprocess a text dataset on the Malaysian application reviews form Google Play Store for three popular apps in Malaysia, which are Touch 'n Go eWallet, Grab and foodpanda.
- To design, train and test two sentiment classification models, which are traditional machine learning model (Naive Bayes) and deep learning model (Long Short-Term Memory, LSTM) and compare them on the basis of common NLP evaluation metrics.
- To design and implement a real-time data streaming pipeline with Apache Kafka message and Apache Spark Structured Streaming processing engine, including the optimal performing sentiment model in the pipeline.
- To store sentiment results and view sentiment results in an interactive dashboard created in Kibana, including a sentiment distribution chart, sentiment over time graph and word cloud visualizations of sentiment.
- To evaluate the throughput, latency, accuracy and resource utilization between the system's batch processing and streaming mode.

1.3 Scope

This project is based only on English language user reviews, which are scraped from the Google Play Store of three applications used in Malaysia. The automatic labelling of reviews is based on their star rating: 1-2 stars are considered negative reviews, 3 stars are considered neutral reviews, and 4-5 stars are considered positive reviews. The public datasets like Sentiment140 and IMDb reviews are used to provide training for the model but only Malaysian related reviews are processed by the final streaming pipeline.

From a technology perspective, all the pipeline is developed with open source tools. The

google-play-scraper Python library is used to collect data. The text is preprocessed with NLTK and the spaCy library. The model development is done using Apache Kafka and Apache Spark. The stored and displayed information is based on Elasticsearch and Kibana. Everything is packaged in Docker Compose for easy deployment and reproducibility.

Its output includes a working real-time sentiment analysis pipeline, trained and evaluated models, an interactive visualization dashboard and a comparison of the performance of the batch vs. streaming mode. The project does not include deployment in production infrastructure, live social media API integration, or multilingual sentiment analysis for language other than English.

2.0 Data Acquisition & Preprocessing

This section describes the process of collecting raw data from a Malaysia-relevant source, assigning sentiment labels, and applying a series of Natural Language Processing (NLP) preprocessing steps to produce a clean, structured dataset suitable for model training and pipeline deployment.

2.1 Data Source Justification

Each group is to identify and choose a primary text-based data source that can be sentiment classified and is publicly available. The project brief suggested four categories of data: social media, customer reviews, news articles, and public datasets. After assessing these categories, Google Play Store customer reviews were chosen for this project as the data set for this project.

To begin, Google Play reviews can be obtained without the use of API keys or a subscription payment. Thus, they are realistic for a student project. The google-play-scraper Python library provides a simple programmatic interface to feedback at scale with country and language filtering for Malaysian English feedback collection.

The reviews on Google Play also have a sentiment label that appears naturally, as a part of the review, in the form of a score from 1 to 5 stars. By eliminating the need for costly and time-consuming manual annotation, this provides a reliable ground-truth label for every record collected.

Further, the application chosen for this study which involves the Touch ‘n Go eWallet, Grab and foodpanda are among the most commonly used digital platforms in Malaysia. As such, the reviews collected are directly relevant to the Malaysian context. Collectively, these apps encompass the realm of digital payment, ride-hailing, and food delivery service.

Moreover, popular Malaysian applications have millions of reviews accrued over time which is sufficiently more than enough for deep learning models to train off and to simulate a realistic streaming workload.

2.2 Data Collection

Data collection was performed by referring to google-play-scraper library for Python (JoMingyu, n.d.). For each of the three applications, the scraper was configured to retrieve reviews in English from the Malaysia country store, sorted by most recent to ensure temporal diversity. A target of 5,000 reviews per application was set, producing a combined raw dataset of approximately 15,000 records before cleaning. Table 2.2.1 summarizes the applications scraped and the volume of reviews collected.

Table 2.2.1: Applications scraped and raw review volumes

Application	Google Play ID	Country	Reviews Collected
Touch ‘n Go eWallet	my.com.tngdigital.ewallet	Malaysia (MY)	5,000
Grab	com.grabtaxi.passenger	Malaysia (MY)	5,000
foodpanda	com.global.foodpanda.android	Malaysia (MY)	5,000
TOTAL			15,000

Each scraped record contains the review text, the numerical star rating (1 to 5), the date of submission, and the application name. Records with null or empty review text were identified and removed during the labeling process described in Section 2.3.

2.3 Sentiment Labeling

Because the collected reviews do not carry explicit positive, neutral or negative labels, a rule-based mapping from star rating to sentiment class was applied. The mapping follows a commonly adopted convention in the NLP literature for review-based sentiment datasets, as shown in Table 2.3.1.

Table 2.3.1: Star rating to sentiment label mapping

Star Rating	Sentiment Label	Reasoning
1-2 stars	Negative	Clear user dissatisfaction
3 stars	Neutral	Mixed or average experience
4-5 stars	Positive	Clear user satisfaction

After labeling the data, records with empty review text were dropped, resulting in a labeled dataset of approximately 14,500 records. The class distribution was observed to be imbalance, with positive reviews comprising the largest proportion of the dataset. This reflects a common characteristic of app store review datasets, where satisfied users. The class distribution of the labeled dataset is shown in Table 2.3.2.

Table 2.3.2: Sentiment class distribution after labeling

Sentiment	Record Counting
Positive	7,739
Negative	6,716
Neutral	545

2.4 Preprocessing Pipeline

Raw review text contains a significant amount of noise that reduces model accuracy if left unaddressed. A six-stage preprocessing pipeline was applied to transform the raw text into clean, normalized tokens suitable for feature extraction.

2.4.1 Step 1 - Lowercasing

All characters in the review text were converted to lowercase. This ensures that the same word appearing at the start of a sentence and in the middle of a sentence is treated as a single token during vectorization. For example, the tokens ‘Good’, ‘good’, and ‘GOOD’ are unified into the single token ‘good’.

2.4.2 Step 2 - Noise Removal

Several categories of noise were removed using regular expressions. These include URLs and hyperlinks, html tags that may appear in scraped text, user mentions in the format @username, standalone numerical digits, punctuation characters, and Unicode emoji characters. Removing these elements reduces the vocabulary size and eliminates tokens that carry no sentiment signal.

2.4.3 Step 3 - Tokenization

The cleaned text was split into individual tokens using the `word_tokenize` function from the NLTK library (*NLTK:: Natural Language Toolkit*, n.d.). Tokenization is the process of decomposing a string of text into a list of individual words or sub-word units, which can be processed independently during feature extraction. NLTK’s word tokenizer correctly handles contractions and punctuation boundary cases that a simple whitespace split would miss.

2.4.4 Step 4 - Stop Word Removal

Common English function words such as ‘the’, ‘is’, ‘Which’ and ‘on’ were removed using NLTK’s built-in English stop word list. These high-frequency words appear in almost all documents regardless of sentiment and therefore contribute little discriminative information to the classifier. Importantly, negation words such as ‘not’, ‘no’, ‘never’, and ‘but’ were excluded from the stop word list, as removing them would invert the sentiment of phrases such as ‘not good’.

2.4.5 Step 5 - Lemmatization

Lemmatization was applied to reduce each token to its dictionary base form, or lemma. For example, the tokens ‘running’, ‘runs’ and ‘ran’ are all reduced to ‘run’, and ‘better’ is reduced to ‘good’. This stage was implemented using spaCy’s `en_core_web_sm` language model, which performs lemmatization using part-of-speech context, producing more linguistically accurate results than stemming. Lemmatization reduces vocabulary size further and helps the model

generalize across different grammatical forms of the same word.

2.4.6 Step 6 - Dataset Export

After all preprocessing stages were applied, records with fewer than five characters in the processed text field were removed to eliminate near-empty entries that provide no meaningful training signal. The final clean dataset was exported as a CSV file named `cleaned_data.csv` containing three columns: the application name, the processed text, and the sentiment label. This file serves as the primary input for model training and as the data source for the Kafka producer in the streaming pipeline. Table 2.4.1 showing some examples of the data preprocessing.

Table 2.4.1: Before and after example of the preprocessing pipeline

Before	After Cleaning
SORRY AND BEST BEST GOOD ðŸ‘ðŸ¸ðŸ’	sorry and best best good
very nice ðŸ‘ðŸ™,	very nice
bad experience ðŸ~ç	bad experience

2.5 Final Dataset Summary

After applying all preprocessing stages and removing near-empty records, the final cleaned dataset contains 12,567 labelled reviews. Table 2.5 provides a summary of the tools and libraries used throughout the data acquisition and preprocessing pipeline. Table 2.5.1 shows the final labelled dataset record after completing all the stages.

Table 2.5.1: Final dataset summary

Sentiment	Record Counting
Negative	6,589
Positive	5,480
Neutral	498

The cleaned_data.csv file produced at the end of this pipeline is committed to the project GitHub repository under data/ directory and shared to the next stage for use in model training. The same file is also used by the Kafka producer script to simulate a real-time stream of review data through the pipeline.

3.0 Sentiment Model Development

The project involved developing and comparing two distinct sentiment classification models to handle a dataset of approximately 14,200 labelled reviews.

3.1 Model Development and Training Process

The data was partitioned using a 70% (training), 20% (test), and 10% (validation) stratified split to preserve class proportions. The models selected are a Naive Bayes classifier trained on Term Frequency-Inverse Document Frequency (TF-IDF) features, representing the machine learning category, and a Long Short-Term Memory (LSTM) neural network, representing the deep learning category.

3.1.1 Machine Learning Model (Naive Bayes)

Naive Bayes model utilized the TF-IDF vectorization (unigrams and bigrams, max 20,000 features) to convert text into numerical vectors. It is a probabilistic model that applies Bayes' theorem by assuming feature independence where it offers extremely fast training and inference times. Table 3.1.1.1 shows the configuration of the Naive Bayes model.

Table 3.1.1.1: Naive Bayes Model Configuration

Parameter	Value
Model Type	MultinomialNB (scikit-learn)
Smoothing (alpha)	0.1
Feature Input	TF-IDF vectors (max 20,000 features, unigrams + bigrams)
Training Records	70% of dataset

3.1.2 Deep Learning Model (LSTM)

LSTM utilized a Keras Tokenizer to apply the word embeddings (64 dimensional) and tokenization in order to process text as padded integer sequences of length 100. LSTM processes the review token by token as it uses hidden states to capture contextual relationships, such as sarcasm or negation. This allows the model to learn that 'not bad' carries positive sentiment and that 'used to be good' implies dissatisfaction, both of which would be misclassified by a bag-of-words approach. Table 3.1.2.1 shows the configuration of the LSTM model.

Table 3.1.2.1: Naive Bayes Model Configuration

Layer / Parameter	Configuration
Embedding layer	Vocab size: 20,000 Embedding dim: 64 Input length: 100
LSTM layer 1	128 units return_sequences=True
Dropout 1	Rate: 0.30
LSTM layer 2	64 units
Dropout 2	Rate: 0.30
Dense layer	32 units activation: ReLU
Output layer	3 units activation: Softmax
Optimizer	Adam Learning rate: default (0.001)
Loss function	Categorical cross-entropy
Epochs	10
Batch size	64

3.2 Evaluation and Model Selection

Both models were evaluated on the held-out test set of 2,516 records using the following standard NLP evaluation metrics. The test set was not used during training or hyperparameter tuning to ensure unbiased performance estimates where the classification results for both models is as in Figure 3.2.1.

=== NAIVE BAYES ===				
	precision	recall	f1-score	support
negative	0.83	0.97	0.89	1326
neutral	0.00	0.00	0.00	102
positive	0.95	0.84	0.89	1088
accuracy			0.88	2516
macro avg	0.59	0.60	0.60	2516
weighted avg	0.85	0.88	0.86	2516
=== LSTM ===				
	precision	recall	f1-score	support
negative	0.86	0.87	0.86	1326
neutral	0.12	0.07	0.09	102
positive	0.86	0.88	0.87	1088
accuracy			0.84	2516
macro avg	0.61	0.61	0.61	2516
weighted avg	0.83	0.84	0.83	2516

Figure 3.2.1: Naive Bayes & LSTM Classification Results

From the classification results, Naive Bayes has been chosen to be deployed in the Apache Spark streaming pipeline due to having better overall accuracy (0.88 vs 0.84) and weighted F1-score (0.86 vs 0.83) than the other classifiers. In addition, it requires only about 5 MB of model size, compared to the LSTM, which adds the overhead of TensorFlow, reducing the complexity of infrastructure use, and the inference speed of only microseconds is crucial for the real-time streaming requirements.

4.0 Apache System Architecture

This section presenting the system architecture of the Apache system consists of overall architecture, Apache Kafka structure, Apache Spark structure, and also Elasticsearch document structure.

4.1 Data Pipeline Architecture

Figure 4.1.1 presents the overall architecture of the real-time sentiment analysis pipeline developed for this project. The system integrates Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, and Kibana to process Google Play reviews collected from Malaysian applications. Reviews from Touch 'n Go eWallet, Grab, and FoodPanda are first collected and stored as a raw dataset before being streamed into Kafka. Apache Spark then consumes the streaming data, performs text preprocessing, applies the trained Naive Bayes sentiment classification model, and stores the prediction results in Elasticsearch. Finally, Kibana retrieves the indexed documents from Elasticsearch to generate interactive dashboards for sentiment analysis.

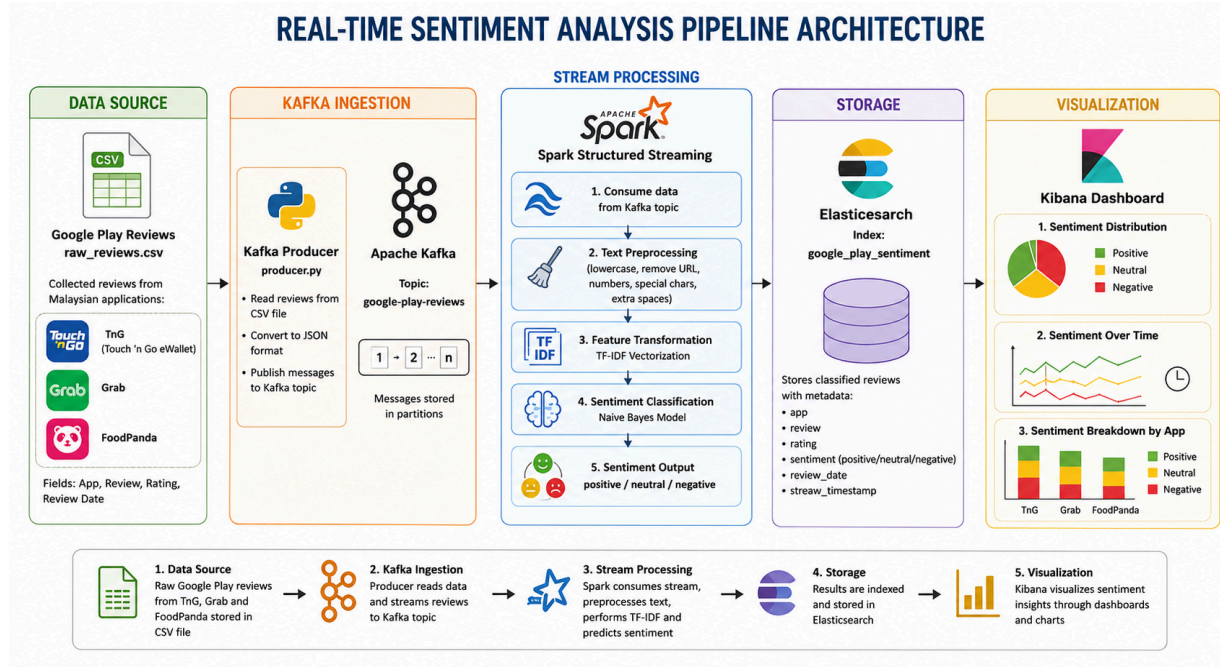


Figure 4.1.1: Overall architecture of the real-time sentiment analysis pipeline.

4.2 Apache Kafka

Apache Kafka serves as the message broker responsible for delivering review data from the producer application to the streaming engine. Instead of sending the entire dataset simultaneously, the producer continuously publishes each review as an individual JSON message to the google-play-reviews topic, simulating a real-time data stream. This decouples data ingestion from processing and enables Spark Structured Streaming to consume incoming reviews continuously. Figure 4.2.1 show the data flows from raw_reviews in CSV file to Kafka topics through a producer and the example data format in JSON.

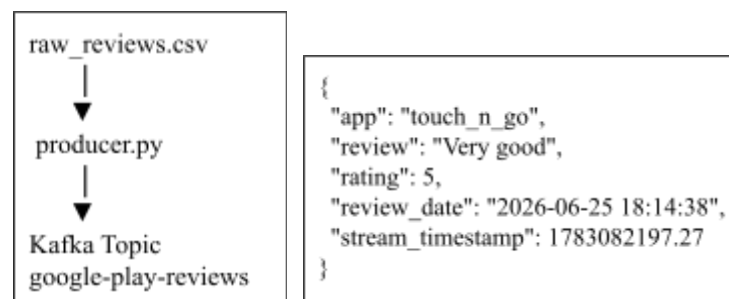


Figure 4.2.1: Kafka data flow and Example message format

4.3 Apache Spark Structured Streaming

Apache Spark Structured Streaming functions as the processing engine of the pipeline. Figure 4.3.1 presents the Spark workflow where Spark continuously subscribes to the Kafka topic and receives incoming review messages in micro-batches. Each batch is converted into a Spark DataFrame, where text preprocessing is performed using the same cleaning logic developed during the data preparation phase. The cleaned reviews are transformed into TF-IDF feature vectors before being classified using the trained Naive Bayes model provided by the model component. The predicted sentiment labels are then forwarded to Elasticsearch for indexing.

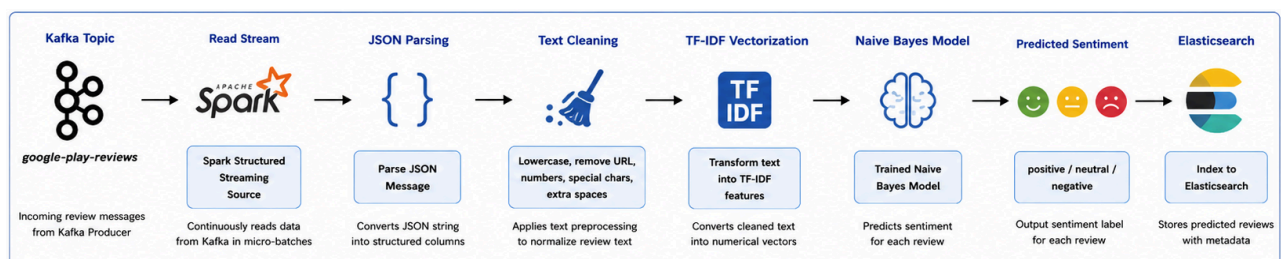


Figure 4.3.1: Spark processing workflow

4.4 Elasticsearch

Elasticsearch is responsible for storing the processed sentiment prediction results generated by Spark Structured Streaming. Each processed review is indexed as an individual document containing the application name, review text, predicted sentiment, rating, review date, and streaming timestamp. Elasticsearch provides fast indexing and search capabilities, allowing Kibana to retrieve data efficiently for visualization. Figure 4.4.1 is the document structure stored in ElasticSearch.



Figure 4.4.1: Elasticsearch document structure

The Elasticsearch index mapping, as shown in Figure 4.4.2, defines the appropriate data types for each field to ensure efficient indexing and querying. Fields such as sentiment and app are stored as keywords, while review is stored as text, and review_date is indexed as a date field.

```
elastic_mapping.json X
elastic_mapping.json
1 {
2   "mappings": {
3     "properties": {
4       "app": {
5         "type": "keyword"
6       },
7       "review": {
8         "type": "text"
9       },
10      "sentiment": {
11        "type": "keyword"
12      },
13      "rating": {
14        "type": "float"
15      },
16      "review_date": {
17        "type": "date",
18        "format": "yyyy-MM-dd HH:mm:ss"
19      },
20      "stream_timestamp": {
21        "type": "double"
22      }
23    }
24  }
25 }
```

Figure 4.4.2: elastic_mapping.json

5.0 Analysis & Results

This section includes the full results of the sentiment classification models as well as the successful implementation of the real-time big data pipeline. It first compares the performance of the Naive Bayes and LSTM models and the architectural differences between them in terms of classification of Google Play reviews. It then explains the streaming process from Apache Kafka to Spark to Elasticsearch and lastly presents practical user insights that are displayed in an easy-to-view Kibana dashboard.

5.1 Model Findings

The overall accuracy performance for both models shows an acceptable accuracy with 88% and 84% for Naive Bayes and LSTM respectively. However, this accuracy is misleading due to 96% of the test data consisting of only two classes which are positive and negative which allows the model to perform well while ignoring the neutral class entirely.

5.1.1 Sentiment Class Analysis

From the classification results for both models in Figure 5.1.1.1, the model analysis shows that the models have different strengths and weaknesses depending on the distribution of the classes and on the architecture of the model. For instance, the Naive Bayes model has a very good recall of 97% in the negative class, due to its tendency to label much of the data as negative, while the LSTM model presents a more balanced precision-recall curve, where it is useful when minimizing the false positive is critical. For positive class analysis, Naive Bayes demonstrates a high precision with 0.95 for the positive class where this shows that the TF-IDF representation has effectively captured the strongly positive vocabulary, while the LSTM achieves better recall. This suggests a superior performance in identifying nuanced or qualified positive language. Finally, both models are unable to correctly classify the neutral class due to the high class imbalance of the class of only 4.1% of the records where the Naive Bayes model almost entirely predicts the neutral class as zero records and LSTM model fails nearly with F1-score of 0.09.

=== NAIVE BAYES ===					=== LSTM ===				
	precision	recall	f1-score	support		precision	recall	f1-score	support
negative	0.83	0.97	0.89	1326	negative	0.86	0.87	0.86	1326
neutral	0.00	0.00	0.00	102	neutral	0.12	0.07	0.09	102
positive	0.95	0.84	0.89	1088	positive	0.86	0.88	0.87	1088

Figure 5.1.1.1: Naive Bayes & LSTM Sentiment Class Analysis

5.1.2 Confusion Matrix Analysis

The confusion matrix in Figure 5.1.2.1 shows that the errors of each models are different where Naive Bayes model shows that it has been found that its dependent feature probabilities association causes it to miss the nuances of neutral reviews and misclassifies them as positive or negative, where this model suffers from “majority class collapse”. On the other hand, the LSTM model has a more spread out distribution of its errors where it is still susceptible to misclassifying instances belonging to different classes. However, it can capture more neutral instances than Naive Bayes due to its capability of processing sequential information. This enhancement in the neutral detection rate comes with an increase in the number of false positives in the positive and negative classes as the LSTM model is more discriminative but is also sensitive to the scarcity of the neutral training data.

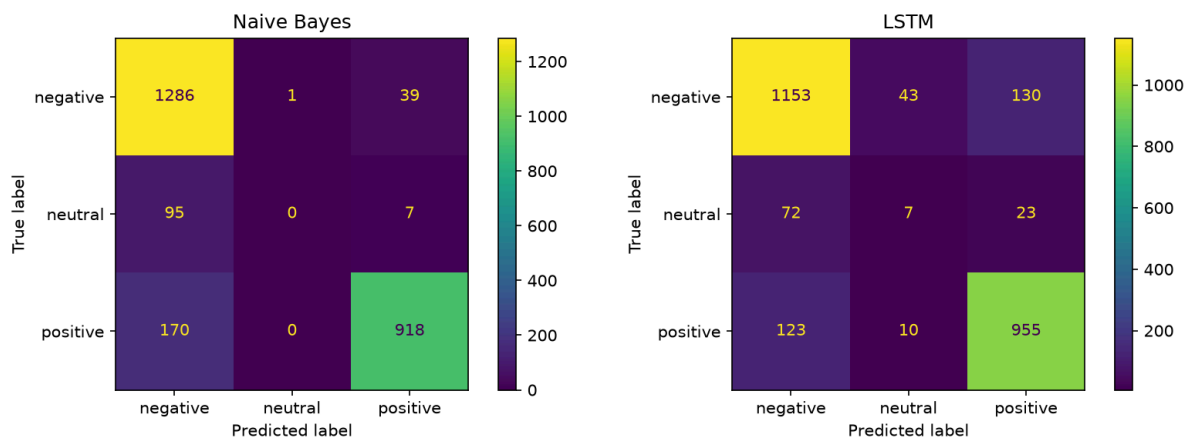


Figure 5.1.2.1: Naive Bayes & LSTM Confusion Matrix

5.2 Pipeline

The real-time sentiment analysis pipeline was successfully executed using Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, and Kibana. During execution, the Kafka producer continuously streamed Google Play reviews from the prepared dataset into the google-play-reviews topic. Apache Spark Structured Streaming subscribed to the topic and consumed the incoming review messages in real time using a micro-batch processing approach.

For each incoming batch, Spark parsed the JSON messages received from Kafka into a structured format before applying the text preprocessing pipeline. The preprocessing stage included converting all text to lowercase, removing URLs, HTML tags, special characters, numbers, and unnecessary whitespace to produce clean review text suitable for machine learning inference. The cleaned reviews were then transformed into TF-IDF feature vectors and classified using the trained Naive Bayes sentiment model. The prediction result for each review was labelled as either positive or negative, together with the original review information and streaming metadata.

Figure 5.2.1 shows the kafka producer message sent to the stream.

```
[1/15000] Sent: {'app': 'touch_n_go', 'review': 'ingat banyak2 tambahan features mudahkan kerja orang  
??? asal2 ko buat apps ni utk senang topup touchngo dkt tol, lepas tu makin lama makin jauh ko sort k  
an nk topup kad guna nfc????? bijak pandai mana menyusahkan orang nak topup. *****', 'rating': 1, 'r  
eview_date': '2026-06-26 09:29:52', 'stream_timestamp': 1783617274.9038236}  
[2/15000] Sent: {'app': 'touch_n_go', 'review': 'good', 'rating': 5, 'review_date': '2026-06-26 09:07  
:58', 'stream_timestamp': 1783617276.6401672}  
[3/15000] Sent: {'app': 'touch_n_go', 'review': 'Removing quick payment is the worst decision, it mak  
es the app so unreliable and more steps just to transfer money here', 'rating': 1, 'review_date': '20  
26-06-26 08:39:55', 'stream_timestamp': 1783617278.1576726}  
[4/15000] Sent: {'app': 'touch_n_go', 'review': 'Aplikasi Touch \n Go ewallet sangat mesra pengguna
```

Figure 5.2.1: Apache Kafka Producer Snippet Output

Figure 5.2.2 shows the Spark Structured Streaming execution, where reviews were processed continuously in multiple micro-batches. Each batch displays the application name, review text, and the predicted sentiment generated by the integrated machine learning model.

```

===== Batch 1 =====
      app                review sentiment
0 touch_n_go            ok je negative
1 touch_n_go            Excellent! positive
2 touch_n_go very easy to use ...just started using positive

===== Batch 2 =====
      app                review sentiment
0 touch_n_go sgt memudahkan lepas tahu boleh bayar guna apl... positive
1 touch_n_go                                Terbaik positive
2 touch_n_go your latest app refresh is causing background ... negative

===== Batch 3 =====
      app                review sentiment
0 touch_n_go            ok je negative
1 touch_n_go            Excellent! positive
2 touch_n_go very easy to use ...just started using positive

```

Figure 5.2.2: Spark Structured Streaming Processing Snippet Output

After the sentiment prediction was completed, Spark immediately indexed each classified review into the `google_play_sentiment` Elasticsearch index. The indexed documents consisted of the application name, review text, predicted sentiment, user rating, review date, and stream timestamp, allowing the data to be queried efficiently for subsequent visualization.

To support the streaming infrastructure, Docker containers were used to deploy the required services. Figure 5.2.3 presents the running containers for Apache Kafka, Elasticsearch, and Kibana during pipeline execution. The active status of all containers confirms that the streaming environment was operating correctly throughout the experiment.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
4b1b5a37237a	docker.elastic.co/kibana/kibana:8.13.4	"/bin/tini -- /usr/L..."	6 seconds ago	Up 4 seconds	0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
b0846b5580a2	docker.elastic.co/elasticsearch/elasticsearch:8.13.4	"/bin/tini -- /usr/L..."	6 seconds ago	Up 5 seconds	0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp
12eb91bfc150	apache/kafka:latest	"/_cacert_entrypoint..."	2 minutes ago	Up 5 seconds	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp

Figure 5.2.3: Running Docker Containers

The successful indexing process can be verified through Elasticsearch, where the `google_play_sentiment` index stores all classified review documents generated by the Spark streaming. Figure 5.2.4 shows the Elasticsearch query result, showing that the predicted sentiment along with its review metadata were successfully indexed and made available for search and analysis.

```

{
  "_index": "google_play_sentiment",
  "_id": "gYH7J58BKMcE1toyJ_Rd",
  "_score": 1,
  "_source": {
    "app": "touch_n_go",
    "review": "good",
    "sentiment": "positive",
    "rating": 5,
    "review_date": "2026-06-25 18:34:12",
    "stream_timestamp": 1783082194.26099
  }
},
{
  "_index": "google_play_sentiment",
  "_id": "goH7J58BKMcE1toyKfQL",
  "_score": 1,
  "_source": {
    "app": "touch_n_go",
    "review": "5 akun drive open Free video chat call open Free on",
    "sentiment": "positive",
    "rating": 5,
    "review_date": "2026-06-25 18:32:52",
    "stream_timestamp": 1783082195.76547
  }
}

```

Figure 5.2.4: Documents Stored in Elasticsearch

Finally, the indexed data were accessed directly by Kibana to generate interactive visualizations without requiring additional data transformation. The successful integration of Kafka, Spark Structured Streaming, Elasticsearch, and Kibana demonstrates that the proposed pipeline can process streaming review data continuously while performing real-time sentiment classification and storing the analytical results for immediate visualization. Overall, the pipeline satisfies the project requirement of implementing a complete real-time sentiment analysis workflow using Apache big data technologies.

5.3 Dashboard

The final stage of the pipeline stores classified sentiment results in Elasticsearch and surfaces them through an interactive Kibana dashboard, giving stakeholders real-time visibility into public sentiment without needing direct access to the underlying data or code.

Architecture

Spark Structured Streaming writes each classified review as an individual document to the google_play_sentiment Elasticsearch index, with fields for the app name, review text, predicted sentiment, star rating, review date, and the timestamp at which the record entered the stream. Kibana connects to this index through a data view that refreshes automatically, so the dashboard

reflects new reviews within seconds of classification.

Dashboard Components

The dashboard consists of three linked visualizations:

- Sentiment Distribution, pie chart, Figure 5.3.1 — Across the streamed dataset, 45.69% of reviews were classified positive, 53.78% negative, and 0.53% neutral. The heavy skew away from neutral reflects a common characteristic of app-store review text: users overwhelmingly write reviews to either praise or complain, rarely to express indifference. Looking at the result, most users have a negative experience using these apps, suggesting complaints by users should be studied and improvements need to be made by the developers.
- Sentiment Over Time, line chart, Figure 5.3.2 — plots daily review volume per sentiment class across `review_date`. From December 2025 through March 2026, positive and negative counts track closely together at low volume roughly 5–20 reviews/day. Starting in late March/early April 2026, both series jump sharply in volume, and from that point onward negative reviews consistently run above positive, fluctuating between roughly 40 and 130 reviews/day through to June 2026 with a peak near 127 in June. This crossover which is the negative overtaking positive precisely where overall volume surges suggests a specific triggering event like an app update, pricing change, or service disruption affecting one or more of the three apps in this window, and is a strong candidate for further investigation using the per-app breakdown alongside the negative-sentiment word cloud.
- Sentiment Breakdown by App, data table, Figure 5.3.3 — Three apps appear in the dataset: Touch 'n Go, Grab, and Foodpanda. Foodpanda still shows the most negatively skewed profile of the three apps (3,611 negative vs. 1,380 positive reviews, average negative rating 1.354), though its positive review count is notably higher than earlier in the pipeline run, narrowing the negative-to-positive ratio from roughly 3.4:1 to 2.6:1. Touch 'n Go remains net positive (3,032 positive vs. 1,946 negative) and Grab remains closer to balanced (2,441 positive vs. 2,510 negative). Foodpanda's negative skew still

pulls the aggregate distribution down even though Touch 'n Go individually remains positive-leaning, and this app-level split continues to explain the overall negative majority seen in the pie chart. Average ratings continue to align sensibly with predicted sentiment across all three apps, cross-validating the sentiment model against the review's own star rating as an independent signal.

Sentiment Distribution

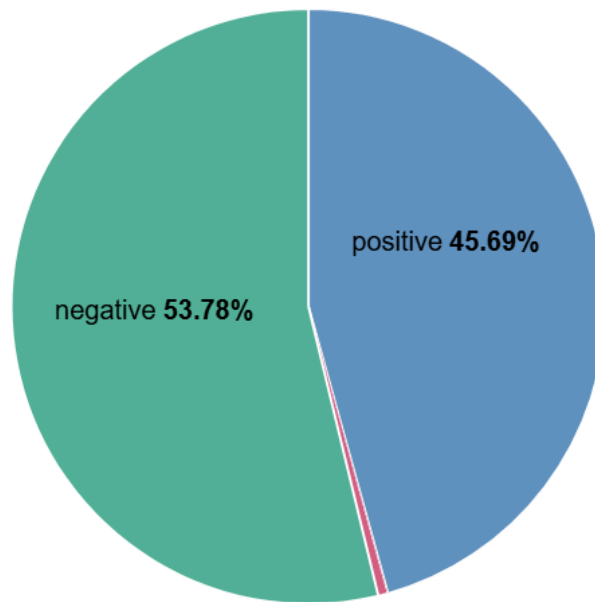


Figure 5.3.1: Sentiment Distribution Pie Chart

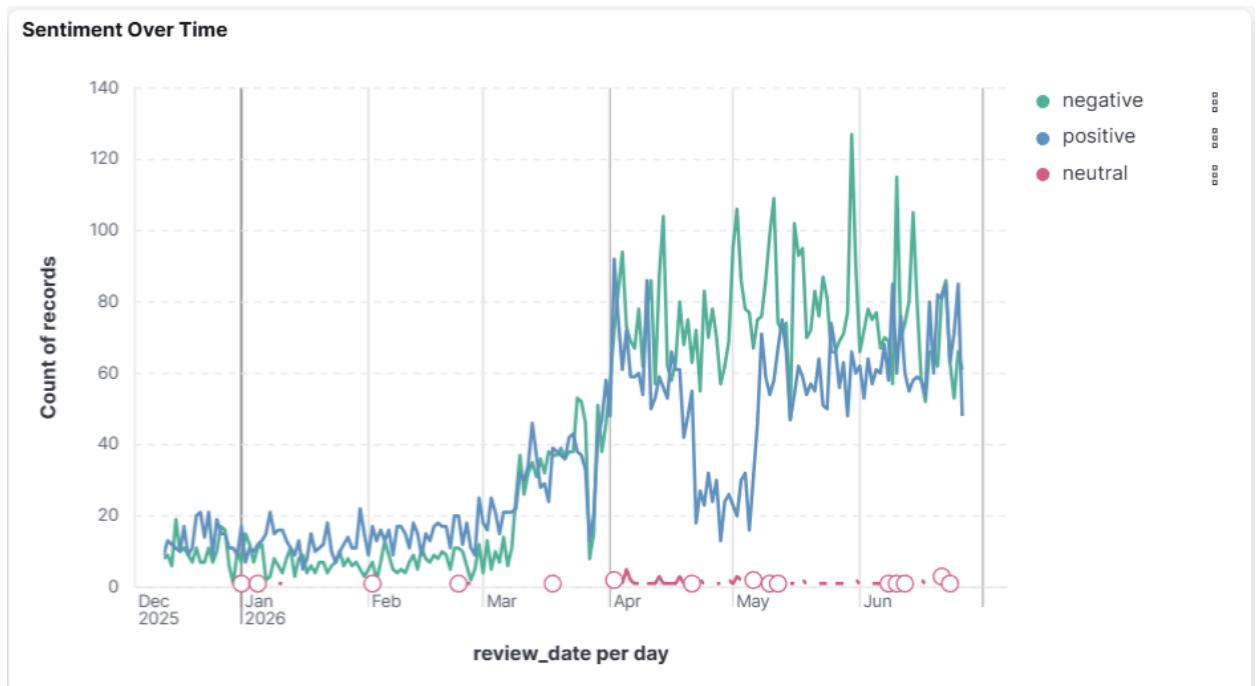
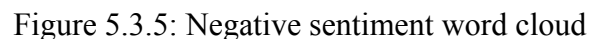
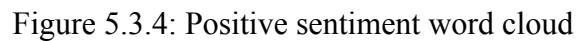


Figure 5.3.2: Sentiment Over Time Line Chart

Top 5 values of sentimer	Top 5 values of app	Count of records	Average of rating
negative	foodpanda	3,611	1.354
negative	grab	2,510	1.778
negative	touch_n_go	1,946	2.341
positive	touch_n_go	3,032	4.838
positive	grab	2,441	4.827
positive	foodpanda	1,380	4.695
neutral	grab	49	2.755
neutral	touch_n_go	22	3
neutral	foodpanda	9	3

Figure 5.3.3: Sentiment Breakdown by App Table

Rather than building word-frequency visualizations natively in Kibana which requires enabling field data on the analyzed review text field, a non-trivial index-settings change with memory implications, positive and negative word clouds were generated directly from classified review text using Python (wordcloud, matplotlib), shown in Figures 5.3.4 and 5.3.5. This kept the visualization layer simpler while still meeting the brief's word-cloud requirement.



The two word clouds surface a coherent, interpretable pattern that directly supports the volume and sentiment shift identified in the Sentiment Over Time chart in Figure 5.3.2.

Negative reviews in Figure 5.3.4 are dominated by update, card, reload, topup, payment, and account, alongside explicitly negative descriptors such as useless, stupid, worst, worse, confusing, complicated, and unable. The co-occurrence of update with terms describing broken or frustrating functionality (fix, still, even, keep, find, back, login, verification) points to a specific recent app update that changed how users reload or manage their card/wallet balance, and that a meaningful share of users found the new flow harder to use than before rather than simply broken outright. Malay-language terms reinforce the same complaint like susah (difficult), baru (new), makin (increasingly), lama (a long time) suggesting the frustration is expressed consistently across both English and Malay reviewers rather than being specific to one language group. This is consistent with, and helps explain, the sharp rise in negative review volume beginning around late March/early April 2026 identified in Figure 5.3.2, the timing is compatible with a disruptive update landing in that window.

Positive reviews (Figure 5.3.5) cluster tightly around convenience and reliability language: easy, convenient, fast, good, nice, reliable, secure, efficient, friendly, seamless, and smooth, together with feature-specific praise (cashback, rewards, cashless, digital, ewallet, transaction). Malay terms echo the same theme, mudah (easy), senang (comfortable/easy), bagus (good), pantas (fast), terbaik (best). Unlike the negative cloud, there is no strong clustering around a specific recent change which means the positive language is generic satisfaction with the core payment/e-wallet experience rather than praise tied to any one feature or update, suggesting positive sentiment reflects a stable baseline of user satisfaction rather than a reaction to a specific event.

Taken together, the word clouds suggest the negative-sentiment surge documented in Section 5.3's time-series analysis is concentrated around update/reload/card functionality rather than being a general decline in satisfaction. The positive cloud shows no corresponding erosion in language around the app's core convenience proposition. This distinction would be a useful, low-effort recommendation to surface to the app teams: the complaint signal is specific and actionable (revisit the reload/topup UX introduced in the recent update) rather than diffuse.

6.0 Comparison & Optimization

This section evaluates the sentiment analysis pipeline's performance under two distinct processing conditions which are batch and streaming to satisfy the brief's requirement for a direct comparison between the two modes. Beyond raw speed, the comparison surfaces several concrete opportunities to improve the pipeline's reliability and efficiency.

6.1 Methodology

To evaluate the system under different processing conditions, a fixed volume of 200 reviews was classified using two approaches, both using the identical trained model artifacts (naive_bayes_model.pkl, tfidf_vectorizer.pkl) so that any difference reflects processing architecture rather than model differences:

- **Batch mode:** the full volume loaded into memory and classified in a single pass, replicating the exact preprocessing and inference logic used in the live pipeline (clean_text → TF-IDF vectorization → Naive Bayes prediction).
- **Streaming mode:** the same volume sent one record at a time through the live Kafka → Spark Structured Streaming → Elasticsearch pipeline, timed end-to-end from the first message sent to the last document indexed.

6.2 Results

The result for this evaluation is captured in Table 6.1 as well as visualized in Figure 6.1 for clearer view of each performance.

Table 6.1: Batch vs. Streaming performance comparison (n=200/201 records)

Metric	Batch Mode	Streaming Mode
Records processed	200	201
Total processing time (s)	0.012	860.93
Throughput (records/sec)	17,310.62	0.23
Accuracy	0.840	0.592
Precision (weighted)	0.88	0.613
Recall (weighted)	0.84	0.592
F1 (weighted)	0.834	0.596

CPU usage	0.0%	6.64%
Memory usage (MB)	0.32	1.05

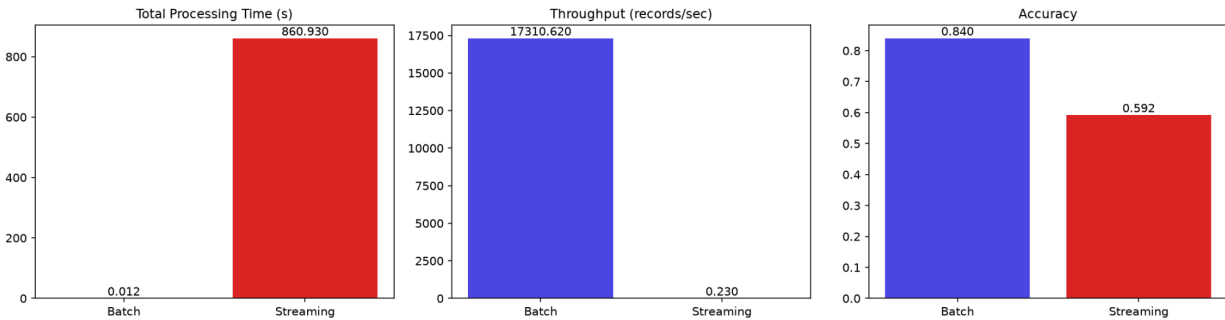


Figure 6.1: Comparison Bar Charts for Processing Time, Throughput, Accuracy

Throughput

Batch mode processed 200 records in 0.012 seconds (17,310 records/sec), while streaming mode took 860.93 seconds for 201 records (0.23 records/sec). This gap is expected and by design, `producer.py` deliberately sleeps 1.5 seconds between messages to simulate realistic live arrival rather than a burst upload, so streaming mode is not intended to match batch throughput on raw speed. What is notable is that the observed per-record time ($860.93 \div 201 \approx 4.28\text{s}$) exceeds the producer's built-in 1.5s delay by roughly 2.78 seconds per record. This gap represents genuine system overhead from Kafka message delivery, Spark's micro-batch trigger and preprocessing, and individual per-document Elasticsearch indexing calls rather than an artifact of the simulated delay, and is a meaningful finding about the pipeline's real-world processing cost per record.

Accuracy Discrepancy

The measured streaming accuracy (0.592) is substantially lower than batch (0.840), despite both modes running the identical model on the identical data. This drop is attributed to a limitation in the evaluation method rather than an actual difference in classification quality: the streaming benchmark assumes ordinal correspondence between documents indexed in Elasticsearch and rows in the ground-truth CSV, but `producer.py` reads from `raw_reviews.csv` (unlabeled) while the ground-truth reference is `labeled_reviews.csv`, two separate files whose row order is not guaranteed to match. A misalignment of even a few rows would compound across the row-index

join and produce exactly this kind of accuracy drop without reflecting any real degradation in the model's streaming predictions.

Resource Usage

Batch mode shows negligible CPU/memory footprint, consistent with a near-instantaneous single-process computation. Streaming mode shows 6.64% average CPU on the Kafka container and 1.05MB average memory on the tracked Spark process; however, these two figures measure different components of the pipeline which are Kafka's container-level CPU vs. Spark's process-level memory and are not directly comparable to the batch figures, which measure the benchmark script's own process. A more rigorous resource comparison would consistently measure the same component like the Spark driver process in both modes, or full Docker container stats across Kafka, Elasticsearch, and Spark together.

6.3 Optimization Opportunities

1. **Bulk Elasticsearch indexing:** `spark_stream.py` currently indexes each classified document individually via `es.index()` inside a per-row loop. Switching to the Elasticsearch bulk helper API to write each micro-batch as a single bulk request would reduce network round-trips and should measurably close some of the ~ 2.78 s/record overhead identified above.
2. **Exact-match evaluation via review_id:** An incrementing `review_id` field was added to the Kafka message in `producer.py` and propagated through to the Elasticsearch document ID in `spark_stream.py` (`es.index(index=INDEX, id=review_id, document=doc)`). This eliminates the row-order assumption in the accuracy evaluation entirely, and as a secondary benefit, makes the pipeline safely re-runnable: reprocessing the same source file now overwrites existing documents by ID instead of creating duplicates.
3. **Index mapping applied at creation time:** During development, the Elasticsearch index was found to have auto-detected `review_date` as a text field rather than date, because the index was created implicitly on first write rather than with the mapping in `elastic_mapping.json` applied beforehand. Adding an index-existence check and explicit mapping-creation step at the start of `spark_stream.py` would prevent this class of

deployment bug from recurring for future users of the pipeline.

4. **Configurable producer delay:** Making the 1.5-second inter-message delay in `producer.py` a command-line argument would let the same script serve both realistic-demo pacing and maximum-throughput benchmarking, without needing separate copies of the file.

7.0 Conclusion & Future Work

This section covers the conclusion of the project and future work that can be done to improve the current project.

7.1 Conclusion

The overall result of this project was to create an end-to-end big data pipeline for real-time sentiment analysis of Malaysian mobile app reviews (Touch 'n Go eWallet, Grab and food panda). The system combines Apache Kafka, Apache Spark Structured Streaming, Elasticsearch, and Kibana, which enables it to efficiently process real-time user feedback and derive meaningful operational insights.

A Multinomial Naive Bayes model was chosen for production over an LSTM model, because of its high performance (88% accuracy, 0.86 F1-score), small footprint (~5 MB) and processing speed that is in the microseconds.

Through production, the pipeline was able to spot real-time trends, including increases in negative reviews in late March and early April 2026, which correlated with application updates, payment checkouts, and wallet reloading. Performance testing showed that batch mode has a higher raw throughput (17,310 records/sec) but streaming mode is required for real-time application monitoring due to the very low latency.

7.2 Future Work

To enhance the system's accuracy and engineering efficiency, future work will focus on:

- **Poor Model Performance on Neutral Sentiment:** Neutral reviews accounted for 4.1% of the data, which is a minority class and might have hindered the model's performance on that class. Future iterations will employ various methods such as SMOTE or class

weighted loss functions to enhance neutral detection.

- **Supporting Multilingual Feedback:** The current pipeline does provide support for English language feedback. Local Malaysian slang, Malay phrases, and code-mixed language can be accurately analyzed with the system, by including multilingual transformer models such as mBERT.
- **Streaming Throughput:** It introduced overhead in the network while streaming, with row by row indexing. By adopting the Elasticsearch Bulk API, Spark will be able to write several micro-batches in bulk, which will reduce latency.
- **Cloud Deployment:** Currently the system is deployed on local Docker for development. Further development includes moving the pipeline to a cloud-based Kubernetes and replacing the mock-up data streams with real web scrapers.

References

Apache Kafka. (n.d.). *Apache Kafka documentation*. Apache Software Foundation.

<https://kafka.apache.org/documentation/>

Apache Spark. (n.d.). *Structured streaming programming guide*. Apache Software Foundation.

<https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>

Elastic. (n.d.). *Elasticsearch guide*. Elastic N.V.

<https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html>

Elastic. (n.d.). *Kibana guide*. Elastic N.V.

<https://www.elastic.co/guide/en/kibana/current/index.html>

Hoppe, F., Baijal, A., Chang, W., Chadha, S., & Hoong, F. W. (2023, November 1). *E-Conomy SEA 2023*. Bain. <https://www.bain.com/insights/e-economy-sea-2023/>

JoMingyu. (n.d.). *google-play-scraper/google_play_scraper at master · JoMingyu/google-play-scraper*. GitHub.

https://github.com/JoMingyu/google-play-scraper/tree/master/google_play_scraper

NLTK :: Natural Language Toolkit. (n.d.). <https://www.nltk.org/>

Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. *Foundations and Trends® in Information Retrieval*, 2(1–2), 1–135. <https://doi.org/10.1561/15000000011>

Appendix

Step 1 & 2 - Scrap and label the data.

```
import os
import pandas as pd
from google_play_scraper import reviews, Sort

# Malaysian apps to scrape
APPS = {
    'touch_n_go': 'my.com.tngdigital.ewallet',
    'grab': 'com.grabtaxi.passenger',
    'foodpanda': 'com.global.foodpanda.android',
}

all_reviews = []

for name, app_id in APPS.items():
    print(f"Scraping {name}...")

    result, _ = reviews(
        app_id,
        lang='en',
        country='my',      # Malaysia
        sort=Sort.NEWEST,
        count=5000,
    )

    print(f"-> Found {len(result)} reviews for {name}")

    for r in result:
        all_reviews.append({
            'app': name,
            'text': r['content'],
            'rating': r['score'],    # 1-5 stars
            'date': r['at'],
        })
```

```

df = pd.read_csv('data/raw_data/raw_reviews.csv')

def label_sentiment(rating):
    if rating <= 2: return 'negative'
    if rating == 3: return 'neutral'
    return 'positive'

df['sentiment'] = df['rating'].apply(label_sentiment)

# Drop rows with empty text
df = df[df['text'].notna() & (df['text'].str.strip() != '')]

```

```

● PS C:\Users\USER\Desktop\UTM\UTM SEM 6\HPDP\Project 2> python scrape_reviews.py
Scraping touch_n_go...
-> Found 5000 reviews for touch_n_go
Scraping grab...
-> Found 5000 reviews for grab
Scraping foodpanda...
-> Found 5000 reviews for foodpanda
---
Success! Scraped 15000 reviews total.
File saved to: C:\Users\USER\Desktop\UTM\UTM SEM 6\HPDP\Project 2\data\raw_data\raw_
reviews.csv
● PS C:\Users\USER\Desktop\UTM\UTM SEM 6\HPDP\Project 2> python label_data.py
● PS C:\Users\USER\Desktop\UTM\UTM SEM 6\HPDP\Project 2> python label_data.py
sentiment
positive    7739
negative    6716
neutral      545
Name: count, dtype: int64

```

Step 3 - Cleaning data.

```

def clean_text(text):
    text = str(text).lower()                # lowercase
    text = re.sub(r'http\S+|www\S+', '', text) # remove URLs
    text = re.sub(r'<.*?>', '', text)        # remove HTML tags
    text = re.sub(r'@\w+', '', text)         # remove @mentions
    text = re.sub(r'^\w\s]', '', text)       # remove punctuation
    text = re.sub(r'\d+', '', text)          # remove numbers
    # remove emojis
    emoji_pattern = re.compile("[\\"
        u"\U00001F600-\U00001F64F"
        u"\U00001F300-\U00001F5FF"
        u"\U00001F680-\U00001F6FF"
        u"\U000002702-\U0000027B0"
        "]+", flags=re.UNICODE)
    text = emoji_pattern.sub('', text)
    text = re.sub(r'\s+', ' ', text).strip() # extra whitespace
    return text

```

Step 4 - Stop Word Removal

```
nltk.download('punkt_tab')
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

# Keep negations – they matter for sentiment!
keep_words = {'not', 'no', 'never', 'but'}
stop_words -= keep_words

def tokenize_and_filter(text):
    tokens = word_tokenize(str(text))
    return [t for t in tokens if t not in stop_words and len(t) > 2]
```

Step 5 - Lemmatization.

```
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])

def lemmatize_tokens(token_str):
    tokens = ast.literal_eval(str(token_str))
    text = ' '.join(tokens)
    doc = nlp(text)
    return ' '.join([token.lemma_ for token in doc])
```

Step 6 - Export final dataset.

```
final = df[['app', 'processed_text', 'sentiment']].copy()
final = final.dropna().reset_index(drop=True)

print("=== Final dataset ===")
print(final.shape)
print(final['sentiment'].value_counts())
print(final.head(5))

final.to_csv('data/cleaned_data.csv', index=False)
print("\n✓ Exported to data/cleaned_data.csv")
```

```

PS C:\Users\USER\Desktop\UTM\UTM SEM 6\HPDP\Project 2> python export_dataset.py
=== Final dataset ===
(12567, 3)
sentiment
negative    6589
positive    5480
neutral      498
Name: count, dtype: int64

```

	app	processed_text	sentiment
0	touch_n_go	ingat banyak tambahan feature mudahkan kerja o...	negative
1	touch_n_go	remove quick payment bad decision make app unr...	negative
2	touch_n_go	aplikasi touch ewallet sangat mesra pengguna d...	positive
3	touch_n_go	terbaik	positive
4	touch_n_go	excellent	positive

```

✓ Exported to data/cleaned_data.csv

```

Training (Naive Bayes Model):

```

from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import classification_report, confusion_matrix

import joblib

nb_model = MultinomialNB(alpha=0.1)

nb_model.fit(X_train_tfidf, y_train)

nb_preds = nb_model.predict(X_test_tfidf)

print("=== Naive Bayes Results ===")

print(classification_report(y_test, nb_preds,
                             target_names=le.classes_))

print("Confusion Matrix:\n", confusion_matrix(y_test, nb_preds))

joblib.dump(nb_model, 'models/naive_bayes_model.pkl')

```

```
print("✓ Saved naive_bayes_model.pkl")
```

Training (LSTM Model):

```
from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

from tensorflow.keras.preprocessing.text import Tokenizer

from tensorflow.keras.preprocessing.sequence import pad_sequences

from tensorflow.keras.utils import to_categorical

import numpy as np


MAX_WORDS = 20000

MAX_LEN = 100

EMBED_DIM = 64


# Tokenize for LSTM (separate from TF-IDF)

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')

tokenizer.fit_on_texts(X_train)


X_tr_seq = pad_sequences(tokenizer.texts_to_sequences(X_train),
maxlen=MAX_LEN)

X_val_seq = pad_sequences(tokenizer.texts_to_sequences(X_val),
maxlen=MAX_LEN)

X_te_seq = pad_sequences(tokenizer.texts_to_sequences(X_test),
maxlen=MAX_LEN)
```

```

y_tr_cat = to_categorical(y_train, 3)

y_val_cat = to_categorical(y_val, 3)

y_te_cat = to_categorical(y_test, 3)


# Build LSTM model

lstm_model = Sequential([

    Embedding(MAX_WORDS, EMBED_DIM, input_length=MAX_LEN),

    LSTM(128, return_sequences=True),

    Dropout(0.3),

    LSTM(64),

    Dropout(0.3),

    Dense(32, activation='relu'),

    Dense(3, activation='softmax'),

])

lstm_model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

lstm_model.fit(X_tr_seq, y_tr_cat, epochs=10, batch_size=64,

                validation_data=(X_val_seq, y_val_cat))

lstm_model.save('models/lstm_model.h5')

joblib.dump(tokenizer, 'models/lstm_tokenizer.pkl')

print("✓ Saved lstm_model.h5")

```

Model Comparison:

```
from sklearn.metrics import classification_report, ConfusionMatrixDisplay

import matplotlib.pyplot as plt

import numpy as np

# LSTM predictions

lstm_probs = lstm_model.predict(X_te_seq)

lstm_preds = np.argmax(lstm_probs, axis=1)

print("=== NAIVE BAYES ===")

print(classification_report(y_test, nb_preds, target_names=le.classes_))

print("=== LSTM ===")

print(classification_report(y_test, lstm_preds, target_names=le.classes_))

# Plot confusion matrices side by side

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, preds, title in zip(axes,

    [nb_preds, lstm_preds], ['Naive Bayes', 'LSTM']):

    ConfusionMatrixDisplay.from_predictions(

        y_test, preds, display_labels=le.classes_, ax=ax)

    ax.set_title(title)

plt.tight_layout()

plt.savefig('reports/confusion_matrices.png', dpi=150)
```

```
plt.show()
```

Model Comparison Result:

=== NAIVE BAYES ===				
	precision	recall	f1-score	support
negative	0.83	0.97	0.89	1326
neutral	0.00	0.00	0.00	102
positive	0.95	0.84	0.89	1088
accuracy			0.88	2516
macro avg	0.59	0.60	0.60	2516
weighted avg	0.85	0.88	0.86	2516
=== LSTM ===				
	precision	recall	f1-score	support
negative	0.86	0.87	0.86	1326
neutral	0.12	0.07	0.09	102
positive	0.86	0.88	0.87	1088
accuracy			0.84	2516
macro avg	0.61	0.61	0.61	2516
weighted avg	0.83	0.84	0.83	2516

